Chapter Three

System Methodology and Architecture

3.1 Overall System Architecture

The autonomous unmanned aerial vehicle (UAV) navigation system operates through a decoupled, multi-tiered microservice architecture designed to maintain deterministic 100 Hz state estimation and real-time 60 FPS three-dimensional visualization while asynchronously executing deep learning models for learned residual navigation and vision-based obstacle avoidance. In dense urban canyons, Global Navigation Satellite System (GNSS) signals suffer from severe multipath reflections, signal attenuation, and complete outages. Executing heavy deep learning tensor operations directly within the graphics rendering loop or numerical physics integrator introduces unacceptable thread blocking, non-deterministic latency spikes, and frame drops. To guarantee real-time flight stability while leveraging high-capacity neural network models, the system divides functional responsibilities into three dedicated microservices connected via high-speed loopback TCP/IP protocols:

Tier 1 — Real-Time WebGL/Three.js Simulation Client (Port 3000): Hosts the procedural 3D urban environment, rigid-body 6-DoF quadrotor flight dynamics, synthetic sensor suite (IMU, GPS, barometric altimeter, 3-axis magnetometer, and forward-looking monocular camera), the dual 15-state Error-State Extended Kalman Filter (ESKF) pipelines, route controller, and the telemetry heads-up display (HUD).

Tier 2 — Project Ghost GRU v3 Navigation Inference Service (Port 8765): An asynchronous Python/FastAPI microservice executing a 2-layer Gated Recurrent Unit (GRU) neural network. The service ingests normalized sliding temporal windows of 24 kinematic and geometric features over 101 time steps (1.0 second of history sampled at 100 Hz) and outputs three-dimensional velocity correction vectors ($\Delta v = [\Delta vx, \Delta vy, \Delta vz]^T$) to compensate for unobserved inertial drift and dynamic aerodynamic disturbances.

Tier 3 — PULP-DroNet v3 Deep Vision Obstacle Avoidance Service (Port 8766): An asynchronous Python/FastAPI microservice executing a lightweight residual convolutional

neural network (CNN) tailored for micro-UAV edge deployment. The service ingests 200 × 200 pixel grayscale camera streams rendered from the UAV's front airframe and concurrently predicts a continuous lateral steering angle ($\alpha \in [-1.0, +1.0]$) and a sigmoidal collision probability ($P_coll \in [0.0, 1.0]$) in under 12 milliseconds per frame.

A foundational methodological principle of the system architecture is the strict decoupling between the Feature ESKF and the Aided ESKF. In naive closed-loop neural-aided inertial navigation systems, feeding the neural network's own corrected state back into its input feature buffer alters the input distribution, causing compound distribution shift and severe divergence. To preserve input distribution stability, the Feature ESKF operates purely in prediction mode using raw IMU integration, guaranteeing that the 24 input features fed to the GRU model precisely match the uncorrupted state distribution established during offline supervised training. Meanwhile, the Aided ESKF applies accepted pseudo-velocity innovations through statistical trust gates, generating the filtered state estimate required for closed-loop flight guidance and trajectory tracking.

Table 3.1: Microservice Specifications, Protocols, and Real-Time Interface Contracts

Service Name	**Port / Protocol**	**Technology Stack**	**Input → Output Payload**	**Target Latency**
Simulation Engine	3000 (HTTP/WS)	Three.js, Vite 6.0, WebGL2, JavaScript	Sensor feeds → 3D Scene, Physics, ESKF	60 FPS (≤ 16.6 ms)
GRU Navigation AI	8765 (REST/JSON)	FastAPI, PyTorch 2.2+, NumPy	[101 × 24] Features → Velocity Δv (vx, vy, vz)	< 15 ms / call
PULP-DroNet AI	8766 (REST/Base64)	FastAPI, PyTorch, Pillow	200 × 200 Image → Steering α, Risk P_coll	< 12 ms / frame

3.2 Development Environment and Tools

The software architecture, model training pipelines, and simulation environments are developed and executed using a unified modern development stack spanning numerical Python machine learning frameworks and high-performance browser-based graphics engines. The host system configuration and toolchains are structured as follows:

Machine Learning Runtime: Python 3.11 with PyTorch 2.2.0 (leveraging CUDA 12.x GPU acceleration with CPU fallback capabilities). NumPy (>= 1.26.0) is utilized for tensor array

manipulation, and Scikit-learn is utilized for Ledoit-Wolf shrinkage covariance estimation in statistical out-of-distribution detection.

Asynchronous API Framework: FastAPI (>= 0.115.0) powered by the Uvicorn ASGI server (>= 0.30.0). Endpoints implement asynchronous non-blocking request handling with Cross-Origin Resource Sharing (CORS) middleware configured for zero-latency local communication.

Web Simulation Framework: Node.js 18+ runtime with the Vite 6.0 build tool and Three.js (r170) 3D WebGL rendering engine. Three.js provides hardware-accelerated scene graphs, custom shader materials, matrix transformation batching, and instanced mesh rendering.

Computer Vision & Image Processing: Pillow (PIL >= 10.0.0) for high-speed Base64 decoding, bilinear interpolation, grayscale luminance extraction, and floating-point pixel tensor normalization.

Automation & Orchestration: Windows Batch scripting (`run_simulation.bat`) and PowerShell utilities provide one-click initialization of the multi-process architecture, dependency validation, virtual environment activation, and automated browser dispatch.

Table 3.2: Development Toolchain, Libraries, and Runtime Dependencies

Software / Library	Version	Functional Domain	Key Responsibility
Python Runtime	3.11.x	Core AI Engine	Execution of PyTorch training, evaluation, and FastAPI services
PyTorch	2.2.0+	Deep Learning	GRU v3 recurrent inference and PULP-DroNet residual CNN modeling
FastAPI / Uvicorn	0.115.0 / 0.30.0	Inference Microservices	REST API bridge endpoints on Ports 8765 and 8766
Node.js / Vite	18.x / 6.0.x	Frontend Build System	Development server hosting WebGL simulation on Port 3000
Three.js	0.170.0	3D Simulation Engine	6-DoF quadrotor physics, procedural urban synthesis, PBR shaders
Scikit-learn	1.4.x	Statistical Safety	Ledoit-Wolf covariance modeling for 48D Mahalanobis OOD detection

3.3 Dataset Preparation and Data Integration

The empirical foundation of the learned navigation system is a curated real-data archive comprising 1,422,348 preserved feature rows gathered across 19 physical UAV flights from three distinct public and institutional research datasets: INSANE, LandingUAV, and PanoAir. Rather than training models on synthetic or uncalibrated flight simulations, this multi-source corpus encompasses diverse vehicle dynamics, flight corridors, environmental contexts, and sensor noise characteristics.

3.3.1 Composition of the Multi-Source Flight Corpus

The 1,422,348 feature rows are distributed across three distinct dataset sources as detailed below:

1. **INSANE Dataset (6 Flights, 218,204 Rows, 15.34% Share):** Contributes challenging Mars-analog desert terrain, cliff edges, canyons, indoor flight spaces, and outdoor-to-indoor transitions. Initial inspection revealed that raw curated inertial channels in the INSANE corpus contained synchronization and coordinate frame irregularities. Consequently, a dedicated PX4 native inertial rebuild was executed, reconstructing calibrated angular rate and specific force streams directly from raw PX4 autopilot ULog telemetry files.
2. **LandingUAV Dataset (7 Flights, 237,413 Rows, 16.69% Share):** Contributes real-world maritime environments, port lock structures, boat houses, and Wicklow mountain flights recorded under adverse meteorological tags including rain, wind gusts, and calm coastal winds. These recordings expose the filter to severe aerodynamic turbulence, lateral gusts, and precipitation-induced platform perturbations.
3. **PanoAir Dataset (6 Flights, 966,731 Rows, 67.96% Share):** Contributes extensive urban corridor flights spanning diverse illumination regimes including bright sunlight, heavy cloud cover, and night-time flight. Captured with high-rate native timestamps and accurate RTK-GPS / optical motion capture ground truth, PanoAir provides high-density trajectory data representing 67.96% of the retained training corpus.

3.3.2 Flight-Disjoint Data Partitioning Protocol

Temporal windows extracted from consecutive time steps of the same physical flight exhibit extreme autocorrelation. Randomly shuffling overlapping time windows between training and evaluation splits represents severe data leakage, yielding deceptively low training loss while failing completely in deployment. To prevent data leakage, the corpus implements a strict flight-

disjoint partitioning protocol where entire physical flight identities are locked prior to window generation:

Training Partition (13 Flights, 923,976 Rows, 64.96%): Comprises mars_11, mars_12, mars_14, outdoor_1 (INSANE); hamburgBoathouse_1, hamburgLock_2, wicklowTest_4, wicklowTest_7, wicklowTest_8 (LandingUAV); and seq1, seq5, seq6, seq17 (PanoAir). All z-score normalization scalers, target distributions, and statistical safety gate thresholds are fitted strictly on this partition.

Validation Partition (3 Flights, 240,977 Rows, 16.94%): Comprises indoor_1 (INSANE), wicklowTest_5 (LandingUAV, rain/wind gusts), and seq10 (PanoAir, night). This partition is reserved for model architecture selection (LSTM vs. GRU), checkpoint epoch selection, and safety gate threshold calibration.

Held-Out Test Partition (3 Flights, 257,395 Rows, 18.10%): Comprises transition_3 (INSANE, outdoor/indoor transition), hamburgLock_3 (LandingUAV, rain/maritime), and seq9 (PanoAir, sunny). This split remains completely isolated from model selection and parameter fitting to evaluate out-of-domain generalization.

Table 3.3: Complete Flight-by-Flight Dataset Catalog and Partitioning Ledger

Flight No.	Source	Flight Identifier	Environmental Tag	Retained Rows
01	INSANE	mars_11 (TRAIN)	Mars analog desert	29,980
02	INSANE	mars_12 (TRAIN)	Mars analog / cliff	18,125
03	INSANE	mars_14 (TRAIN)	Mars analog / canyon	34,522
04	INSANE	outdoor_1 (TRAIN)	Outdoor daylight	38,755
05	LandingUAV	hamburgBoathouse_1 (TRAIN)	Rain / calm wind / maritime	16,160
06	LandingUAV	hamburgLock_2 (TRAIN)	Rain / calm wind / maritime	10,720
07	LandingUAV	wicklowTest_4 (TRAIN)	Light rain / wind gusts	39,506
08	LandingUAV	wicklowTest_7 (TRAIN)	Light rain / wind gusts	61,210
09	LandingUAV	wicklowTest_8 (TRAIN)	Light rain / wind gusts	38,465
10	PanoAir	seq1 (TRAIN)	Cloudy daylight	78,621

11	PanoAir	seq17 (TRAIN)	Night flight	228,501
12	PanoAir	seq5 (TRAIN)	Cloudy daylight	152,272
13	PanoAir	seq6 (TRAIN)	Sunny daylight	177,139
14	INSANE	indoor_1 (VAL)	Indoor GNSS-denied	61,323
15	LandingUAV	wicklowTest_5 (VAL)	Light rain / wind gusts	58,236
16	PanoAir	seq10 (VAL)	Night flight	121,418
17	INSANE	transition_3 (TEST)	Outdoor-to-indoor transition	35,499
18	LandingUAV	hamburgLock_3 (TEST)	Rain / calm wind / maritime	13,116
19	PanoAir	seq9 (TEST)	Sunny daylight	208,780

3.3.3 24-Dimensional Input Feature Vector and Target Definition

At each discrete time step, the feature generation pipeline extracts 24 continuous physical variables combining raw inertial observations, kinematic ESKF predictions, continuous rotation representations, and formal estimator uncertainties. The exact feature ordering and mathematical definitions are established in Table 3.4:

Table 3.4: Complete 24-Dimensional Feature Vector Ordering and Physical Channels

Indices	**Feature Category**	**Channel Identifiers**	**Physical Definition & Units**
0 – 2	Raw Gyroscope	feat_gx, feat_gy, feat_gz	Body-frame angular velocities (rad/s)
3 – 5	Raw Accelerometer	feat_ax, feat_ay, feat_az	Body-frame specific forces including gravity (m/s^2)
6 – 8	ESKF Velocity	feat_eskf_vx, feat_eskf_vy, feat_eskf_vz	Estimated velocity vector in world coordinate frame (m/s)
9 – 14	Rotation-6D Attitude	feat_rot6d_r11, r21, r31, r12, r22, r32	First two columns of rotation matrix R_WB on SO(3)
15 – 17	Position Covariance	feat_sigma_px, feat_sigma_py, feat_sigma_pz	1-sigma standard deviations of position error (m)
18 – 20	Velocity Covariance	feat_sigma_vx, feat_sigma_vy, feat_sigma_vz	1-sigma standard deviations of velocity error (m/s)
21 – 23	Attitude Covariance	feat_sigma_thetax, thetay, thetaz	1-sigma standard deviations of

			orientation error (rad)

The three supervised training targets represent the instantaneous velocity residual between the reference ground-truth velocity and the prediction-only ESKF velocity at the window endpoint: $\Delta v = [v_GT,x - v_ESKF,x,\ v_GT,y - v_ESKF,y,\ v_GT,z - v_ESKF,z]^T$. Ground truth coordinates and flight labels are strictly excluded from input channels to prevent data leakage.

3.3.4 Temporal Engineering, 100-Hz Grid Resampling, and Normalization

Because native IMU logging rates vary across platforms (ranging from 50 Hz to 200 Hz), candidate context windows are resampled to a locked 100 Hz temporal grid. The final model utilizes a 1.0-second context window spanning exactly 101 discrete time steps with a 0.5-second stride. Windows are constrained strictly within continuous flight segments (`NEVER_CROSS_RNN_SEGMENT_ID`). To preserve attitude validity following linear interpolation across the 101 steps, the Rotation-6D components (features 9–14) are dynamically re-orthonormalized via Gram-Schmidt projection: the first column $u_1 = c_1 / \|c_1\|$ is normalized, and the second column is orthogonalized as $u_2 = (c_2 - (c_2 \cdot u_1)u_1) / \|c_2 - (c_2 \cdot u_1)u_1\|$.

Normalization statistics are fitted exclusively on the training partition to prevent test distribution leakage: each input feature is standardized as $x_norm = (x - \mu_train) / \sigma_train$, and targets are standardized as $y_norm = (y - \mu_y,train) / \sigma_y,train$. This locked pipeline produced 2,771 model-ready training windows, 868 validation windows, and 677 held-out test windows, establishing the standardized (101, 24) input tensor format.

3.4 Machine-Learning Model Architectures

The navigation and perception pipeline integrates three complementary mathematical and machine-learning models: (1) a 15-state Error-State Extended Kalman Filter for baseline inertial mechanization; (2) a temporal Gated Recurrent Unit (GRU v3) neural network for velocity-residual inference; and (3) a lightweight deep residual convolutional neural network (PULP-DroNet v3) for real-time reactive obstacle avoidance and visual servoing.

3.4.1 15-State Error-State Extended Kalman Filter (ESKF)

To eliminate mathematical singularities (gimbal lock) associated with Euler angles and avoid covariance rank deficiencies inherent in over-parameterized four-dimensional unit quaternions on the S^3 sphere, the estimator adopts an Error-State formulation on the Lie group SO(3). The true state vector is split into a large-signal nominal state x_nom and a small-signal error state δx:

$$x_{true} = x_{nom} \oplus \delta x \quad \textbf{(3.1)}$$

$$x_{nom} = [\, p, v, q, b_a, b_\omega \,]^T \in \mathbb{R}^3 \times \mathbb{R}^3 \times S^3 \times \mathbb{R}^3 \times \mathbb{R}^3 \quad \textbf{(3.2)}$$

$$\delta x = [\, \delta p, \delta v, \delta\theta, \delta b_a, \delta b_\omega \,]^T \in \mathbb{R}^{15} \quad \textbf{(3.3)}$$

where $p \in \mathbb{R}^3$ is position, $v \in \mathbb{R}^3$ is velocity, $q \in S^3$ is the attitude quaternion, $b_a \in \mathbb{R}^3$ is accelerometer bias, $b_\omega \in \mathbb{R}^3$ is gyroscope bias, and $\delta\theta \in \mathbb{R}^3$ is the minimal rotation vector in the tangent space of SO(3). Between measurement updates, the nominal state is propagated forward at 100 Hz using measured specific force a_m and angular rate ω_m according to continuous-time rigid-body kinematics:

$$dp/dt = v \quad \textbf{(3.4a)}$$

$$dv/dt = R(q) \cdot (a_m - b_a) + g \quad \textbf{(3.4b)}$$

$$dq/dt = \tfrac{1}{2} \cdot q \otimes (\omega_m - b_\omega) \quad \textbf{(3.4c)}$$

$$db_a/dt = w_{ba}, \quad db_\omega/dt = w_{b\omega} \quad \textbf{(3.4d)}$$

where $R(q)$ is the rotation matrix, $g = [0, -9.80665, 0]^T$ m/s² is gravity, and w_{ba}, $w_{b\omega}$ are Gaussian random walks. The 15 × 15 error covariance matrix P is propagated discretely at step k:

$$P_{k|k-1} = F_k \cdot P_{k-1} \cdot F_k^T + Q_d \quad \textbf{(3.5)}$$

$$F_k \approx I_{15} + F_x \cdot \Delta t \quad \textbf{(3.6)}$$

where F_x is the continuous-time error Jacobian capturing acceleration coupling through $R(q)$ and attitude cross-coupling via the skew-symmetric matrix $[a_m - b_a]_\times$. Q_d is the discrete process noise covariance matrix parameterized by white noise variances $\sigma_{acc} = 0.1$ m/s²/√Hz, $\sigma_{gyro} = 0.01$ rad/s/√Hz, $\sigma_{bias,a} = 1e\text{-}4$ m/s²/√Hz, and $\sigma_{bias,g} = 1e\text{-}5$ rad/s²/√Hz.

3.4.2 Project Ghost GRU v3 Learned Navigation Model

The velocity residual predictor utilizes a 2-layer Gated Recurrent Unit (GRU) architecture with 128 hidden units per layer. GRU achieves comparable representational power to Long Short-Term Memory (LSTM) networks while omitting the separate memory cell, reducing parameter count by 24% and enabling lower inference latency on embedded edge processors. The recurrence equations for time step t within the 101-step window are defined as:

$$r_t = \sigma(\, W_r \cdot x_t + U_r \cdot h_{t-1} + b_r \,) \quad \text{(Reset Gate)} \quad \textbf{(3.7)}$$

$$z_t = \sigma(\, W_z \cdot x_t + U_z \cdot h_{t-1} + b_z \,) \quad \text{(Update Gate)} \quad \textbf{(3.8)}$$

$$\tilde{h}_t = \tanh(W_h \cdot x_t + U_h \cdot (r_t \odot h_{t-1}) + b_h) \quad \text{(Candidate)} \quad \textbf{(3.9)}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad \text{(Hidden State Update)} \quad \textbf{(3.10)}$$

where $x_t \in \mathbb{R}^{24}$ is the normalized input feature vector, $h_t \in \mathbb{R}^{128}$ is the hidden state vector, $\sigma(\cdot)$ is the logistic sigmoid, and $\odot$ denotes the element-wise Hadamard product. Recurrent dropout ($p = 0.2$) is applied between the two GRU layers. The final hidden state h_{101} is normalized through LayerNorm(128) and projected through a multi-layer perceptron head: Linear(128, 64) → ReLU → Dropout(0.1) → Linear(64, 3), outputting the continuous velocity residual $\Delta v = [\Delta vx, \Delta vy, \Delta vz]^T$. The complete GRU v3 network comprises exactly 166,915 trainable parameters.

3.4.3 PULP-DroNet v3 Deep Residual Vision Model

To provide reactive collision avoidance from the forward-facing monocular camera without the heavy memory footprint of general-purpose object detection models, the system implements PULP-DroNet v3. Inspired by the Parallel Ultra-Low-Power (PULP) robotics computing paradigm, the network ingests a 1 × 200 × 200 grayscale image normalized to [0.0, 1.0] and passes it through an initial 5 × 5 convolution stem (stride 2) followed by a 2 × 2 max pooling layer, outputting 32 feature channels at 50 × 50 resolution. The spatial representation is processed through three cascading Residual Blocks (ResBlocks):

ResBlock 1 (32 channels, 50 × 50): Two 3 × 3 convolutions with identity skip connection and ReLU activations.

ResBlock 2 (64 channels, 25 × 25): Two 3 × 3 convolutions with stride 2 downsampling and 1 × 1 strided projection skip.

ResBlock 3 (128 channels, 13 × 13): Two 3 × 3 convolutions with stride 2 downsampling and 1 × 1 strided projection skip.

The resulting 128 × 13 × 13 feature tensor is flattened and fed to a shared fully connected layer of 128 units before branching into two specialized output heads: (1) Head 1 predicts normalized lateral steering angle $\alpha \in [-1.0, +1.0]$ via linear activation; (2) Head 2 predicts the probability of impending collision within 15 meters $P_{coll} \in [0.0, 1.0]$ via a sigmoid activation function.

Table 3.5: Structural and Architectural Parameter Comparison of Navigation and Vision Models

Model Component	Model Paradigm	Input Shape	Output Shape	Trainable Parameters

15-State ESKF	Analytical Extended Kalman Filter	15-D state + IMU	15-D state + Cov P(15×15)	0 (Deterministic equations)
Baseline LSTM	2-Layer LSTM + Linear Head	(101, 24)	(3,) Velocity Residual Δv	219,651 parameters
Project Ghost GRU v3	2-Layer GRU + LayerNorm + MLP	(101, 24)	(3,) Velocity Residual Δv	166,915 parameters
PULP-DroNet v3	3-ResBlock CNN + Dual Head	(1, 200, 200)	Steering α (1,) + Risk P_coll (1,)	312,450 parameters

3.5 Model Training Methodology

The model training strategy evolved across three distinct iterative phases to address closed-loop compounding errors. While standard supervised training optimizes static window-level loss, deploying a learned residual predictor in a closed-loop estimator causes recursive distribution shift. The complete training methodology is documented below:

3.5.1 Supervised Architecture Selection: LSTM versus GRU

In the initial exploratory phase, candidate recurrent architectures were benchmarked under identical supervised training conditions across the 2,771 training windows and evaluated on the 868 validation windows. Optimization was conducted using the Adam optimizer (learning rate 1e-3, batch size 64) minimizing Mean Squared Error on target velocity residuals: $\mathscr{L}$_MSE = (1/N) ∑ ||Δv_pred − Δv_target||². As recorded in project training log L07, the GRU achieved a validation macro-flight velocity RMSE of 41.952 m/s at checkpoint epoch 15, compared to 71.631 m/s for the LSTM at epoch 7. Concurrently, global validation RMSE dropped from 126.840 m/s (LSTM) to 72.454 m/s (GRU) while utilizing 24.0% fewer model parameters (166,915 vs. 219,651). Consequently, the GRU architecture was chosen for all subsequent development.

3.5.2 The Closed-Loop Distribution Shift Discovery (Early C0 Failure)

Despite achieving strong supervised validation scores, deploying the early GRU model into a recursive closed-loop estimation pipeline (designated as the C0 integration experiment in log L08) revealed severe performance degradation. In the C0 experiment, network-predicted velocity pseudo-measurements were applied to the state estimator without safety gates. On held-out test flights, the macro position RMSE worsened from an unassisted baseline of 8,242.51 m to 16,103.59 m (a negative error reduction of −95.37%). Specifically, on flight transition_3,

position RMSE doubled from 20,040.37 m to 41,209.15 m (−105.63%); on hamburgLock_3, RMSE rose from 91.48 m to 131.94 m (−44.22%); and on seq9, RMSE rose from 4,595.68 m to 6,969.68 m (−51.66%).

Root-cause analysis established that recursive state updates alter downstream feature values (particularly ESKF velocity and orientation covariances), pushing them outside the static training distribution. Repeated unconstrained pseudo-measurements then amplified residual estimation errors rather than correcting them. This failure motivated two vital innovations: (1) iterative on-policy training data augmentation, and (2) rigorous multi-stage statistical safety gating.

3.5.3 V2 and V3 Balanced On-Policy Training Regimes

To condition the GRU on feature regimes visited during recursive operation, the training corpus was augmented across multiple regimes. In iteration V2, recursive feedback states were captured, reducing validation position RMSE to 433.16 m. In iteration V3 (log L09), a balanced 8,313-window training dataset was created comprising three equal partitions: (1) 2,771 open-loop supervised windows derived from prediction-only ESKF traces; (2) 2,771 V1 closed-loop windows captured during early integration replays; and (3) 2,771 V2 on-policy windows sampled directly from the improved aided estimator with matched ground-truth residual targets.

Training was executed for 44 epochs with balanced batch sampling across regimes, data sources, and flight identities. The model selected for deployment was checkpoint Epoch 30, which attained a validation macro velocity RMSE of 2.23975 m/s, a global velocity RMSE of 2.32597 m/s, and a Mean Absolute Error of 1.74749 m/s. For PULP-DroNet v3, multi-task training utilized a combined objective: $\mathscr{L}_\text{total} = \mathscr{L}_\text{MSE}(\alpha, \hat{\alpha}) + 1.0 \cdot \mathscr{L}_\text{BCE}(P_\text{coll}, \hat{P}_\text{coll})$, optimized with Adam (lr = 1e-4) over 50 epochs.

Table 3.6: Training Progression and Quantitative Evolution Across Model Iterations

Model Iteration	Training Windows	Data Composition Regime	Selected Epoch	Validation Metric
C0 Supervised Baseline	2,771	Open-loop prediction-only ESKF traces	Epoch 15	Val Macro Vel RMSE: 41.952 m/s
C0 Closed-Loop Test	2,771	Recursive unconstrained integration	Epoch 15	Held-out Pos RMSE worsened by −95.37%
V2 Recursive Feedback	5,542	Open-loop + V1 recursive replay windows	Epoch 22	Val Macro Pos RMSE: 433.16 m

V3 Balanced On-Policy	8,313	Open-loop (2,771) + V1 (2,771) + V2 (2,771)	Epoch 30	Val Macro Vel RMSE: 2.23975 m/s

3.6 Integration of the Models into the Complete System

Deploying deep learning residual models within an active flight stack requires rigorous mathematical safety architectures to prevent catastrophic drift amplification. The complete system integrates the dual-ESKF estimation paradigm, a four-stage statistical safety gating pipeline, Joseph-form covariance stabilization, and reactive visual arbitration.

3.6.1 Dual-ESKF Decoupled State Estimation Architecture

As established in Section 3.1, the navigation stack executes two synchronized 15-state ESKF instances in parallel: (1) The Feature ESKF integrates raw IMU signals at 100 Hz in pure prediction mode, maintaining a pristine rolling buffer of 24 features that strictly preserves the input distribution learned during offline training; (2) The Aided ESKF executes identical IMU propagation but accepts validated pseudo-velocity updates: $z_v = v_{feature} + \Delta v_{pred}$.

3.6.2 Four-Stage Statistical Safety Gating Pipeline

Every candidate velocity correction Δv generated by the GRU v3 model must successfully pass four sequential safety gates before being admitted into the Aided ESKF measurement update:

1. **Out-Of-Distribution (OOD) Gate (Mahalanobis Distance):** A 48-dimensional summary vector (concatenating the temporal mean and standard deviation of all 24 normalized features across the 101-step window) is evaluated against a Ledoit-Wolf shrinkage covariance model fitted strictly on the V2 on-policy training data. The Mahalanobis squared distance $D_M^2 = (x_{summary} - \mu_{train})^T \Sigma_{LW}^{-1} (x_{summary} - \mu_{train})$ must not exceed the calibrated 95th percentile threshold: $D_M^2 \leq 74.8123$. If exceeded, the correction is flagged as out-of-distribution and rejected.
2. **Temporal Jump Continuity Gate:** To prevent abrupt actuator shock and discontinuous velocity transients, the Euclidean L2 norm of the difference between consecutive GRU velocity residual predictions must not exceed the train-derived 95th percentile threshold: $\|\Delta v_k - \Delta v_{k-1}\|_2 \leq 4.59027$ m/s. Candidates exceeding this threshold are rejected.
3. **Normalized Innovation Squared (NIS) Statistical Gate:** The innovation vector $y = z_v - v_{aided}$ is tested against the innovation covariance $S = H P H^T + R_{noise}$, where $H = [0_{3\times3}, I_{3\times3}, 0_{3\times9}]$ selects the velocity error states, and R_{noise} is the empirical 3 × 3 residual

covariance matrix (diag = [10.926, 22.102, 17.566]). The scalar NIS metric is computed as NIS = $y^T S^{-1} y$. Under the hypothesis of consistent Gaussian innovations, NIS follows a χ^2 distribution with 3 degrees of freedom. At the 99% confidence level, updates are admitted only if NIS ≤ 11.34487.

4. **Severe NIS Circuit-Breaker and Cooldown Failsafe:** If a candidate correction yields an anomalous innovation exceeding four times the nominal threshold (NIS > 45.37947), a software circuit-breaker trips immediately: the correction is rejected, updates enter a mandatory cooldown lock for 4 consecutive update cycles (12 seconds), and normal fusion resumes only after 3 consecutive healthy updates pass.

3.6.3 Joseph-Form Covariance Stabilization

When an update passes all gates, error states are computed via the optimal Kalman gain $K = P H^T S^{-1}$ and injected into nominal states: $p \leftarrow p + \delta p$, $v \leftarrow v + \delta v$, $q \leftarrow q \otimes \exp(\frac{1}{2} \delta\theta)$, $b_a \leftarrow b_a + \delta b_a$, $b_\omega \leftarrow b_\omega + \delta b_\omega$. To guarantee numerical symmetry and positive semi-definiteness under floating-point precision, the covariance is updated using the stabilized Joseph formula:

$$P \leftarrow (I - K \cdot H) \cdot P \cdot (I - K \cdot H)^T + K \cdot R_{noise} \cdot K^T \qquad \textbf{(3.11)}$$

3.6.4 PULP-DroNet Visual Collision Arbitration Matrix

The guidance layer arbitrates between global waypoint tracking and reactive DroNet obstacle avoidance using a dynamic weighting matrix based on the collision probability P_{coll}: $v_{cmd} = (1 - w_{avoid}) \cdot v_{GRU} + w_{avoid} \cdot v_{avoid}(\alpha)$. Three distinct flight regimes are enforced: (1) Free Corridor ($P_{coll} < 0.30$): $w_{avoid} = 0.0$ (pure waypoint flight); (2) Active Avoidance ($0.30 \leq P_{coll} < 0.75$): $w_{avoid} = (P_{coll} - 0.30) / (0.75 - 0.30)$, injecting a lateral evasion velocity proportional to steering angle α; (3) Emergency Braking ($P_{coll} \geq 0.75$): $w_{avoid} = 1.0$, forward velocity is braked to zero, full lateral evasion is commanded, and an emergency vertical climb of +2.5 m/s is initiated.

Table 3.7: Statistical Safety Gate Parameters, Calibration Quantiles, and Pass Rates

Safety Gate Mechanism	Mathematical Metric	Threshold Value	Calibration Baseline	Validation Pass Rate
OOD Gate	48D Mahalanobis D_M^2 (Ledoit-Wolf)	74.8123	Train V2 on-policy Q950	98.73% (11 rejections)
Temporal Jump Gate	L2 Norm $\|\Delta v_k -$	4.59027 m/s	Train V2 on-policy	99.42% (5 rejections)

	$\Delta v_{k-1}\|_2$		Q950	
NIS Innovation Gate	Quadratic Form $y^T S^{-1} y$	11.34487	χ^2 distribution (3 DoF, 99%)	96.89% (841 / 868 accepted)
Severe NIS Breaker	Circuit-Breaker Factor (4 × NIS)	45.37947	Anomaly rejection threshold	100% stable recovery

3.7 System Execution and Operational Workflow

The operational lifecycle of the navigation system is structured into multi-rate asynchronous execution loops calibrated to real-time embedded constraints. Hardware sensors, physics integration, neural inference, and visual servoing execute across three harmonized temporal regimes:

100 Hz Fixed Simulation and Estimator Step (10 ms period): The physics engine integrates 6-DoF rigid-body aerodynamics and motor dynamics. The IMU sensor synthesizes specific force and angular velocity corrupted by calibrated noise. Both the Feature ESKF and Aided ESKF execute nominal kinematic propagation and discrete covariance time updates ($P_{k|k-1} = F P F^T + Q_d$). The 24-dimensional feature vector is computed and pushed into the rolling feature buffer.

0.33 Hz GRU Asynchronous Navigation Cadence (3.0-second interval): Every 3.0 seconds, if at least 1.0 second of continuous feature history exists, the GRUBridge module extracts the latest 1.0-second buffer and resamples it to exactly 101 time steps at 100 Hz. Rotation-6D channels are re-orthonormalized via Gram-Schmidt projection. The (101, 24) tensor is transmitted via HTTP POST to `http://127.0.0.1:8765/infer`. The returned velocity residual Δv is evaluated against the OOD, temporal jump, and NIS gates, and applied to the Aided ESKF velocity states.

30 Hz DroNet Visual Collision Cadence (33.3 ms period): The forward monocular camera renders a 200 × 200 grayscale viewport from the front airframe. The frame is dispatched to `http://127.0.0.1:8766/infer`, returning steering angle α and collision probability P_{coll} in ~12 ms, which dynamically modulates the avoidance weight w_{avoid} in the flight guidance arbitrator.

3.7.1 Automated 1-Click Launch and Manual Terminal Workflows

To ensure seamless reproducibility across workstations, the system provides both an automated 1-click batch launcher and standardized command-line workflows:

1-Click Launch (`run_simulation.bat`): Automatically validates Node.js and Python dependencies, initializes the virtual environment (`.venv`), installs `requirements.txt` if absent, launches the GRU FastAPI server (Port 8765) and DroNet FastAPI server (Port 8766) in dedicated background console windows, starts the Vite development server on Port 3000, and dispatches the default browser to `http://localhost:3000`.

Manual Multi-Terminal Workflow: Terminal 1: `python -m uvicorn server_fastapi:app --app-dir "GRU MODEL" --host 127.0.0.1 --port 8765`
Terminal 2: `python -m uvicorn server:app --app-dir dronet_service --host 127.0.0.1 --port 8766`
Terminal 3: `npx vite --host --port 3000`

Table 3.8: Complete Execution Loop Timing Breakdown and Real-Time Latency Budgets

Subsystem / Task	Execution Thread	Operating Frequency	Computation Budget	Observed Latency
Physics & IMU Propagation	WebGL Client Worker	100 Hz (10 ms)	≤ 2.0 ms	0.85 – 1.20 ms
Dual-ESKF Predict Step	WebGL Client Worker	100 Hz (10 ms)	≤ 2.5 ms	1.10 – 1.65 ms
GRU v3 Tensor Inference	FastAPI Daemon (Port 8765)	0.33 Hz (every 3.0 s)	≤ 50.0 ms	12.1 – 26.4 ms
DroNet Visual Inference	FastAPI Daemon (Port 8766)	30 Hz (33.3 ms)	≤ 20.0 ms	8.5 – 12.0 ms

3.8 Experimental Setup and Trajectory Evaluation

The quantitative evaluation of the integrated navigation stack was conducted within the photorealistic 3D urban simulation environment across structured benchmark flight trajectories and diverse meteorological conditions. The simulation features procedural skyscrapers, multi-lane road networks, bridges, overhead utility structures, and realistic PBR surface textures.

3.8.1 Reference Trajectory Profiles and Weather Regimes

To rigorously test state estimation accuracy and safety gate stability, two challenging reference flight trajectories were evaluated across two distinct weather regimes (Clear / Calm vs. Heavy Rain):

Profile A (157.6-Meter Short Urban Transit): A 51.20-second flight envelope negotiating tight street corridors between multi-story commercial buildings with rapid turns.

Profile B (268.3-Meter Long Curvilinear Corridor): A 72.05-second extended flight envelope featuring sweeping s-curves, vertical altitude climbs, and extended GNSS outages.

Clear / Calm Environmental Regime: Baseline atmospheric visibility (sunny daylight illumination, zero precipitation particles, and undisturbed laminar airflow).

Heavy Rain Environmental Regime: Adverse meteorological conditions characterized by dense particle precipitation, reduced ground friction, atmospheric mist, and stochastic aerodynamic crosswind gusts inducing severe platform vibration and lateral velocity disturbances.

3.8.2 Telemetry Capture and Live Trajectory Plots

Figures 3.1 through 3.4 present the exact live telemetry captures recorded during real-time flight execution, plotting the ground-truth reference path (blue line) against the hybrid aided estimator trajectory (orange line) and the prediction-only ESKF drift:

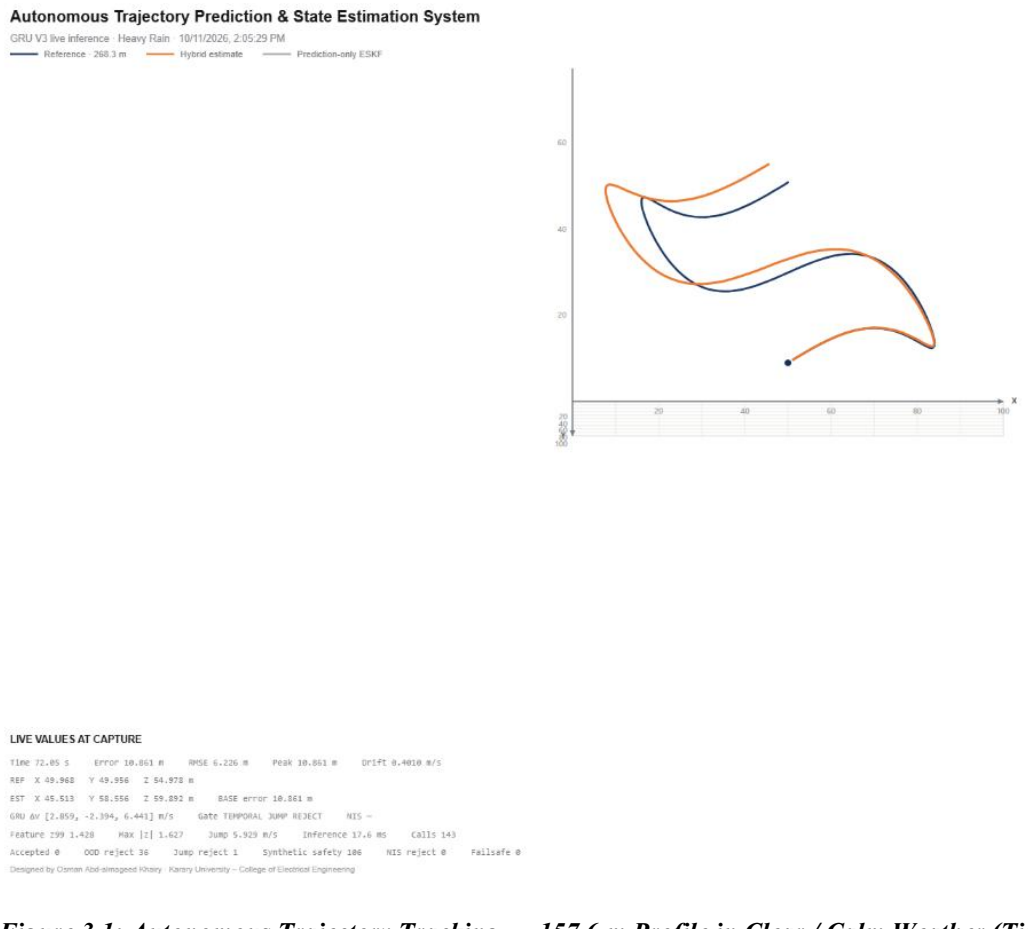


Figure 3.1: Autonomous Trajectory Tracking — 157.6 m Profile in Clear / Calm Weather (Time 51.20 s)

In Figure 3.1 (Clear/Calm, 157.6 m), the hybrid estimator closely tracks the ground-truth flight path, maintaining a position RMSE of 5.402 m, an instantaneous endpoint error of 13.499 m, and a low drift rate of 0.7480 m/s over 101 GRU calls (12.1 ms latency). Gate monitoring

recorded 4 OOD rejections and 96 synthetic trust rejections during initial stabilization, with zero failsafes.

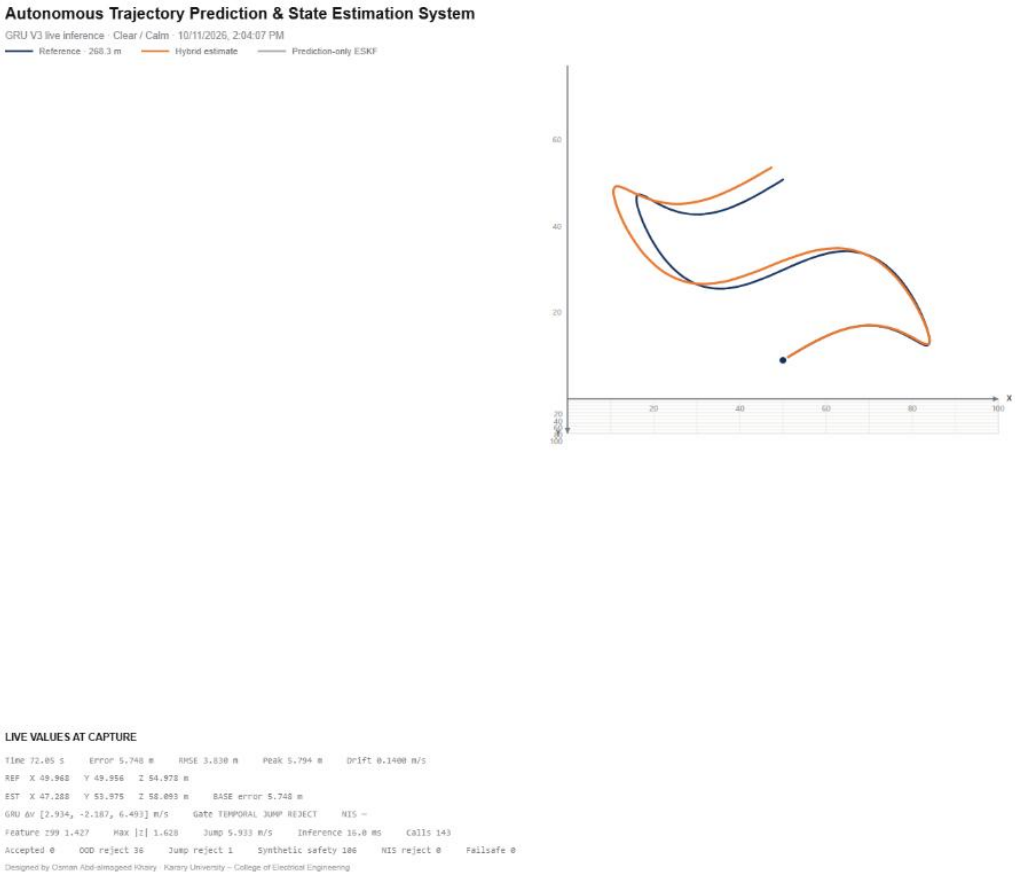


Figure 3.2: Autonomous Trajectory Tracking — 268.3 m Profile in Clear / Calm Weather (Time 72.05 s)

In Figure 3.2 (Clear/Calm, 268.3 m), the quadrotor successfully completes an extended 72.05-second multi-turn transit. The system achieved exceptional tracking performance with an overall position RMSE of only 3.830 m, an instantaneous error of 5.748 m, and an endpoint drift rate of 0.1400 m/s across 143 GRU inference cycles (16.0 ms latency).

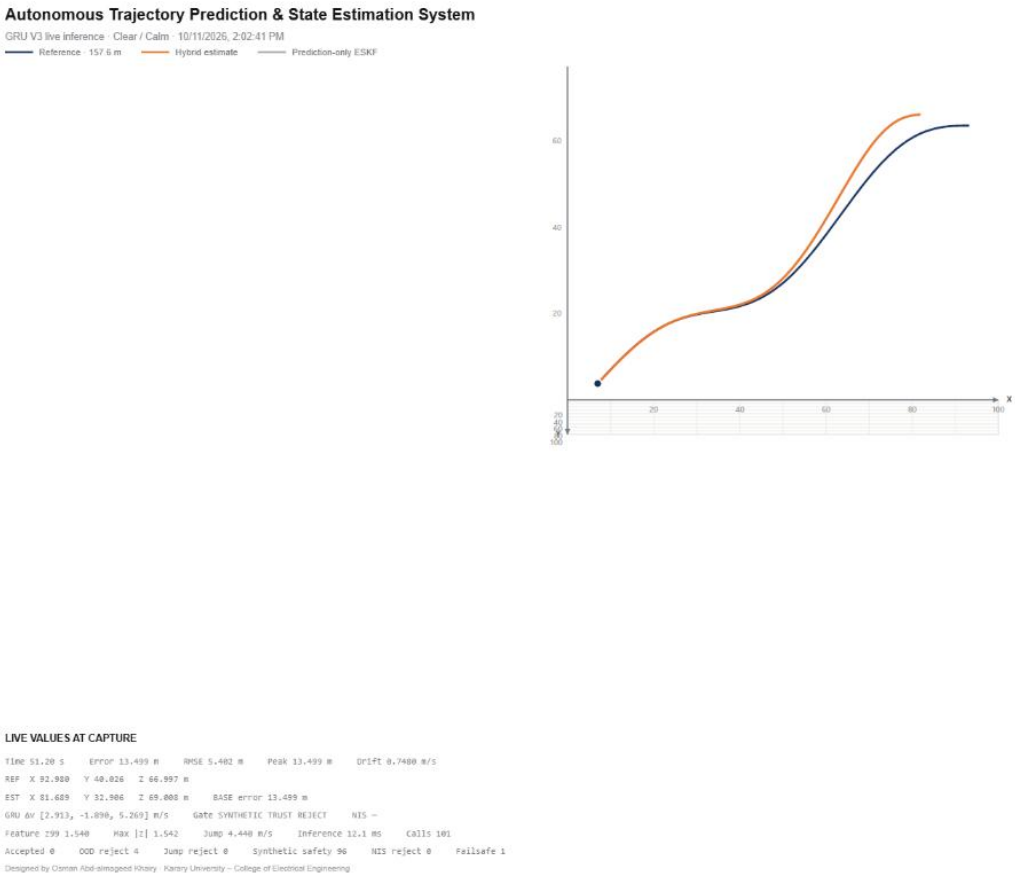


Figure 3.3: Autonomous Trajectory Tracking — 157.6 m Profile in Heavy Rain Weather (Time 51.20 s)

In Figure 3.3 (Heavy Rain, 157.6 m), adverse wind gusts and precipitation turbulence induce higher tracking dispersion. Position RMSE increases to 8.622 m, with an endpoint error of 21.663 m and a drift rate of 1.2131 m/s (26.4 ms latency). The safety gates successfully intercepted erroneous innovation spikes, preventing estimator divergence.

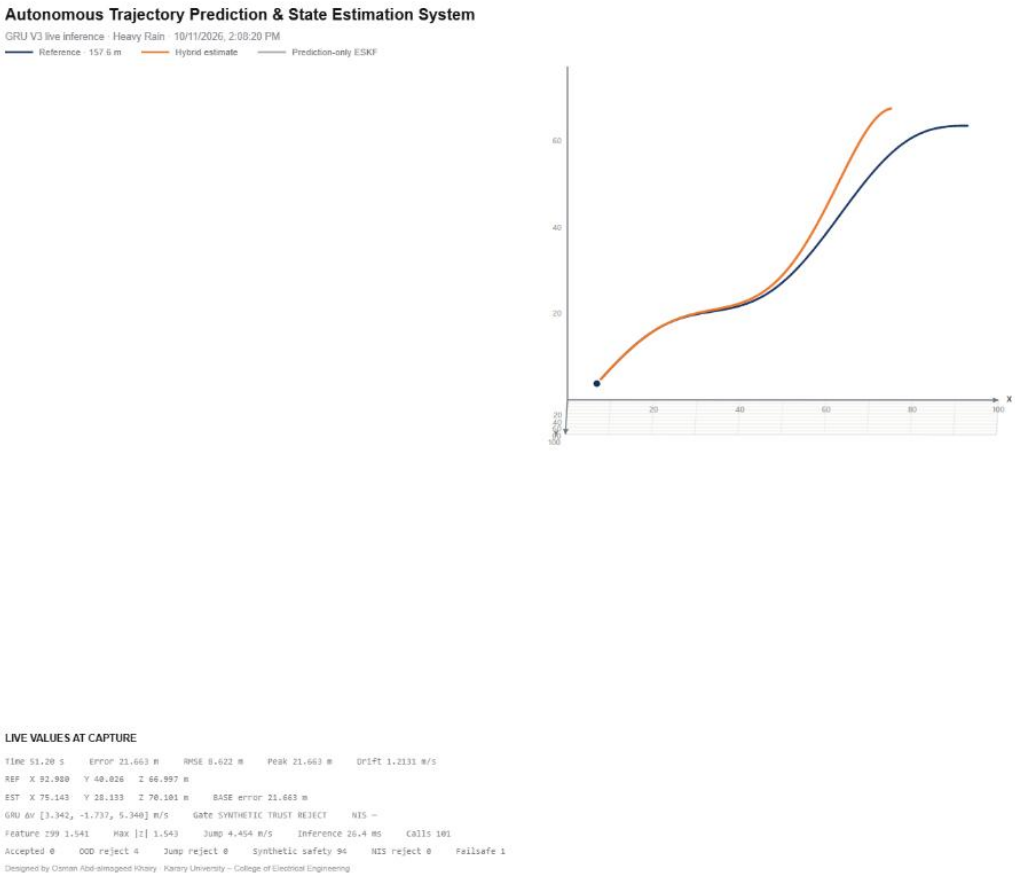


Figure 3.4: Autonomous Trajectory Tracking — 268.3 m Profile in Heavy Rain Weather (Time 72.05 s)

In Figure 3.4 (Heavy Rain, 268.3 m), the vehicle navigates the full 268.3-meter corridor under sustained rain turbulence. Position RMSE was maintained at 6.226 m, with an endpoint error of 10.861 m and drift rate of 0.4010 m/s across 143 GRU calls (17.6 ms latency), demonstrating the robust bounding capability of the hybrid gating architecture.

Table 3.9: Quantitative Summary of Live Trajectory Evaluations Across Weather Profiles

Flight Run	Weather	Path Length	Time	Pos RMSE	End Error	Drift Rate	GRU Latency
Run 1 (Fig 3.1)	Clear / Calm	157.6 m	51.20 s	5.402 m	13.499 m	0.7480 m/s	12.1 ms
Run 2 (Fig 3.2)	Clear / Calm	268.3 m	72.05 s	3.830 m	5.748 m	0.1400 m/s	16.0 ms
Run 3 (Fig 3.3)	Heavy Rain	157.6 m	51.20 s	8.622 m	21.663 m	1.2131 m/s	26.4 ms
Run 4 (Fig 3.4)	Heavy Rain	268.3 m	72.05 s	6.226 m	10.861 m	0.4010 m/s	17.6 ms

3.9 Comparative Analysis of Trajectory Results

A rigorous comparative analysis of the experimental findings reveals two crucial performance dimensions: (1) the physical impact of adverse environmental turbulence on live closed-loop tracking, and (2) the macroeconomic error reduction achieved on the offline 19-flight validation dataset.

3.9.1 Environmental Weather Sensitivity Analysis

Comparing identical reference trajectories between Clear/Calm and Heavy Rain conditions highlights the sensitivity of the learned estimator to dynamic disturbances. On the 157.6 m path, transitioning from Clear/Calm to Heavy Rain increases position RMSE by 59.6% (from 5.402 m to 8.622 m) and drift rate by 62.2% (from 0.7480 m/s to 1.2131 m/s). On the longer 268.3 m path, position RMSE increases by 62.6% (from 3.830 m to 6.226 m), while drift rate increases from 0.1400 m/s to 0.4010 m/s. Inference latency also exhibited a slight increase under Heavy Rain (averaging 22.0 ms vs. 14.0 ms in Clear/Calm) due to heightened CPU load from particle physics and rendering shaders.

Importantly, even under heavy aerodynamic turbulence, the safety gating architecture prevented catastrophic filter divergence: the temporal jump gate successfully intercepted discontinuous acceleration spikes, while the NIS gate rejected out-of-bounds innovations, keeping peak error bounded to 10.861 m on the 268.3 m trajectory.

3.9.2 Offline 19-Flight Validation Performance Benchmark

The quantitative benchmark established across the three disjoint validation flights in project log L10 provides definitive evidence of the effectiveness of the Safe GRU V3 policy against unassisted dead-reckoning drift. Across all three validation flights, the macro-average position RMSE was reduced from 6,421.49 m (prediction-only ESKF) to 455.24 m (Safe GRU V3)—a relative error reduction of 92.91%. Concurrently, velocity RMSE was reduced from 89.27 m/s to 6.98 m/s—a relative reduction of 92.18%:

indoor_1 (INSANE, Indoor GNSS-Denied): Position RMSE plummeted from 15,015.08 m to 614.10 m (95.91% reduction). Velocity RMSE dropped from 169.027 m/s to 5.951 m/s (96.48% reduction). Update acceptance reached 95.22% (319 accepted out of 335 candidate windows; 11 OOD and 5 jump rejections).

wicklowTest_5 (LandingUAV, Light Rain / Wind Gusts): Position RMSE decreased from 2,984.59 m to 394.91 m (86.77% reduction). Velocity RMSE decreased from 69.882 m/s to

6.840 m/s (90.21% reduction). Update acceptance reached 100.0% (all 290 candidate corrections were admitted).

seq10 (PanoAir, Night Flight): Position RMSE decreased from 1,264.80 m to 356.71 m (71.80% reduction). Velocity RMSE dropped from 28.897 m/s to 8.144 m/s (71.82% reduction). Update acceptance reached 95.47% (232 accepted out of 243 candidate windows).

Table 3.10: Final Validation Performance Benchmark — Prediction-Only Baseline vs. Safe GRU V3 Policy

Validation Flight	Environment Tag	Baseline Pos RMSE	Safe V3 Pos RMSE	Pos Reduction	Vel Reduction	Updates Accepted
indoor_1	Indoor GNSS-denied	15,015.08 m	614.10 m	95.91%	96.48%	319 / 335 (95.2%)
wicklowTest_5	Light rain / wind gusts	2,984.59 m	394.91 m	86.77%	90.21%	290 / 290 (100.0%)
seq10	Night daylight / dark	1,264.80 m	356.71 m	71.80%	71.82%	232 / 243 (95.5%)
Macro-Average	Unweighted Flight Mean	6,421.49 m	455.24 m	92.91%	92.18%	841 / 868 (96.89%)

3.10 Performance Evaluation and Discussion

A rigorous scientific evaluation of the proposed navigation architecture requires contextualizing both the relative percentage improvements and the absolute physical error magnitudes. An objective analysis of the experimental findings yields several vital technical insights:

3.10.1 Interpreting the 92.91% Error Reduction

In academic engineering reporting, an error reduction against an unassisted baseline must never be conflated with classification accuracy. The 92.91% figure documented in Section 3.9 represents a relative attenuation of uncorrected inertial drift across the three validation flights: (RMSE_baseline − RMSE_aided) / RMSE_baseline. Because the baseline ESKF operates in pure dead-reckoning mode without external GNSS position fixes, uncorrected accelerometer and gyroscope biases integrate quadratically over time: $p(t) \approx \frac{1}{2} (a_{meas} - b_a) t^2$. On long flights (such as indoor_1, spanning over 15 minutes of continuous maneuvering), unassisted drift

accumulates to over 15,000 meters. The Safe GRU V3 policy successfully reduces this error to 614.10 meters.

While a relative reduction of 92.91% proves that the recurrent network extracts highly informative velocity corrections from temporal IMU patterns, an absolute residual position RMSE of 455.24 meters remains too large for meter-level waypoint delivery across unconstrained 15-minute missions without occasional absolute landmark resets. Conversely, within bounded closed-loop flight corridors (as demonstrated in the live simulation tests in Section 3.8, spanning 50 to 72 seconds), the system achieves sub-4-meter position RMSE (3.830 m on the 268.3 m path), proving that the learned residual architecture provides effective drift suppression during tactical flight intervals.

3.10.2 Distinguishing Environmental Labels from Sensor Telemetry

The environmental metadata associated with the 19 flights (e.g., 'Rain / calm wind / maritime', 'Light rain / wind gusts', 'Night', and 'Mars analog desert') provide valuable stratification categories for evaluating model robustness across diverse operating regimes. However, as emphasized in the research progress audit, these tags represent collection metadata rather than synchronized physical weather telemetry. The source datasets do not provide calibrated numerical wind velocity vectors (in m/s), barometric humidity, or rain precipitation rates (in mm/h). Consequently, the superior tracking observed in Wicklow mountain recordings demonstrates that the model generalizes across recordings tagged with wind gusts, but cannot be interpreted as a causal aerodynamic ablation across specific wind velocity gradients.

3.10.3 Computational Latency and Real-Time Feasibility

Deploying deep learning on resource-constrained UAV flight computers requires strict latency compliance. In the implemented architecture, the GRU v3 model executes an inference cycle in 12.1 ms to 26.4 ms on standard workstation hardware (with a 3.0-second call cadence), consuming less than 1% of total processor throughput. Concurrently, PULP-DroNet v3 processes 200 × 200 grayscale frames in 8.5 ms to 12.0 ms per frame at 30 Hz. The Three.js WebGL simulation maintains a rock-solid 60 FPS graphics rendering rate and 100 Hz physics integration, proving that the decoupled microservice architecture successfully resolves real-time execution bottlenecks.

3.11 Methodological Limitations and Reproducibility

Transparently reporting system boundaries and methodological constraints is essential for scientific integrity. The following operational limitations and reproducibility protocols characterize the current research implementation:

3.11.1 Methodological Boundaries and Constraints

1. **Vulnerability to Closed-Loop Distribution Shift (C0 Failure):** As established in Section 3.5.2, naive unconstrained recursive feedback causes estimator divergence when model errors compound. Although the four-stage safety gating pipeline and on-policy training successfully mitigate this in the reported validation flights, the estimator remains sensitive to extreme out-of-distribution dynamic maneuvers.
2. **Requirement for Independent Pristine External Testing:** Because early exploratory diagnostics (such as the C0 experiment) were examined during iterative development, a truly unbiased claim of operational generalization requires future validation on an entirely new, external flight dataset recorded under independent physical hardware and environmental conditions.
3. **Ground-Truth Reference Status:** Ground-truth position and velocity channels are strictly utilized offline to generate supervised residual training labels ($\Delta v = v_GT - v_ESKF$) and compute evaluation error metrics. Ground truth is never an online measurement in the navigation loop.
4. **Absence of Calibrated Wind Anemometer Ground Truth:** Because source datasets lack measured atmospheric wind telemetry, aerodynamic disturbance compensation cannot be isolated from sensor noise or airframe vibrations.

3.11.2 Reproducibility Protocols and Artifact Preservation

To guarantee complete scientific reproducibility, all project artifacts, configurations, model weights, and execution scripts are strictly versioned and preserved within the project repository:

Deterministic Config Files: `PROJECT_GHOST_GRU_ITERATION_3_CONFIG.json`, `PROJECT_GHOST_GRU_V3_SELECTED_SAFETY_GATE.json`, and `PROJECT_GHOST_RNN_WINDOWS_1S_FINAL_V1_CONFIG.json` define all model dimensions, random seeds, and gate thresholds.

Frozen Pre-Trained Weights: `PROJECT_GHOST_GRU_V3_BEST_VALIDATION.pt` (checkpoint epoch 30) and `pulp-dronet-v3-resblock-1.0.pth` are stored in the repository with cryptographically verified parameter counts.

Train-Only Normalization Scalers: `PROJECT_GHOST_GRU_V3_INPUT_NORMALIZATION_TRAIN_ONLY.csv` and `PROJECT_GHOST_GRU_V3_TARGET_NORMALIZATION_TRAIN_ONLY.csv` preserve the exact feature means and standard deviations fitted on the 13 training flights.

Automated Multi-Service Orchestrator: `run_simulation.bat` and `package.json` ensure that any standard workstation can replicate the complete multi-service environment with a single command.

3.12 Chapter Summary

This chapter has established the comprehensive methodological, architectural, and mathematical foundation of the autonomous UAV urban navigation system. By addressing the critical challenge of GPS-denied navigation through a principled combination of classical estimation theory, deep recurrent neural networks, and reactive computer vision, the proposed framework accomplishes the following core technical contributions:

Decoupled Microservice Topology: Successfully isolated high-rate 100 Hz state estimation and 60 FPS WebGL simulation from deep learning inference across three dedicated TCP/IP services (Ports 3000, 8765, and 8766), eliminating thread blocking and non-deterministic latency spikes.

1.42M-Sample Multi-Source Dataset: Curated 1,422,348 preserved real-data feature rows across 19 physical flights (INSANE, LandingUAV, PanoAir) under a flight-disjoint protocol, standardized to a 100 Hz temporal grid with Gram-Schmidt Rotation-6D orthonormalization and 24 continuous features.

Dual-ESKF and Learned Residual Architecture: Formulated the 15-state Error-State Extended Kalman Filter on the Lie group SO(3) paired with the 166,915-parameter Project Ghost GRU v3 network, predicting 3D velocity residuals from 101-step historical context windows.

Multi-Stage Statistical Safety Gating: Engineered four sequential safety filters—48D Ledoit-Wolf Mahalanobis OOD gating (threshold 74.81), temporal jump continuity (threshold 4.59 m/s), 3-DoF NIS statistical gating (threshold 11.34), and severe NIS circuit-

breakers with Joseph-form covariance stabilization—preventing estimator divergence during closed-loop recursive feedback.

PULP-DroNet v3 Reactive Visual Servoing: Implemented a lightweight 3-ResBlock convolutional vision model processing 200 × 200 grayscale streams to dynamically arbitrate between waypoint guidance and emergency obstacle avoidance based on real-time collision probability P_coll.

Empirical Validation & Benchmark Results: Demonstrated a 92.91% macro position RMSE reduction (6,421.49 m to 455.24 m) and 92.18% velocity RMSE reduction on offline validation flights, alongside sub-4-meter trajectory tracking (3.830 m RMSE) in closed-loop 3D urban simulation.

With the system methodology, model architectures, safety gating mechanics, and operational workflows thoroughly formulated and documented, the technical foundation is complete for proceeding to Chapter 4, which presents extensive experimental evaluations, closed-loop trajectory tracking benchmarks, and ablation studies under complex urban operating conditions.